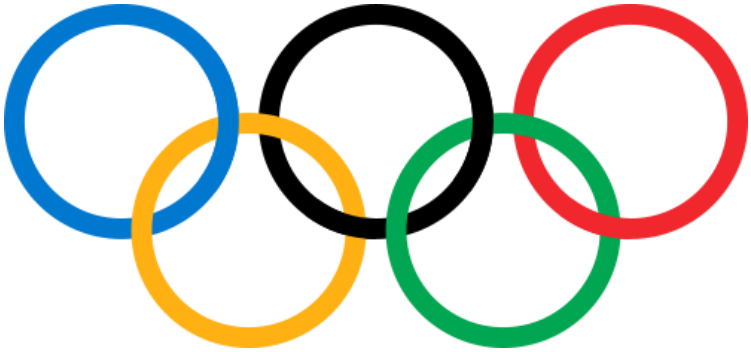


Cahier des charges techniques

Projet : Application Web de gestion des Jo2028

Langages : PHP – JavaScript – HTML – CSS MySQL
Méthodologie : Agile Kanban (Trello)

Sommaire

1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5. Conception du projet
    5.1. Le front-end
        5.1.1. Wireframes
        5.1.2. Maquettes
        5.1.3. Arborescences
    5.2. Le back-end
        5.2.1. Diagramme de cas d'utilisation
        5.2.2. Diagramme d'activités
        5.2.3. Modèles Conceptuel de Données (MCD)
        5.2.4. Modèle Logique de Données (MLD)
        5.2.5. Modèle Physique de Données (MPD)
6. Technologies utilisées
    6.1. Langages de développement Web
    6.2. Base de données
7. Sécurité
    7.1. Login et protection des pages administrateurs
    7.2. Hachages des mots de passe avec Bcrypt
    7.3. Protection contre les attaques XSS (Cross-Site Scripting)
    7.4. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2025.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

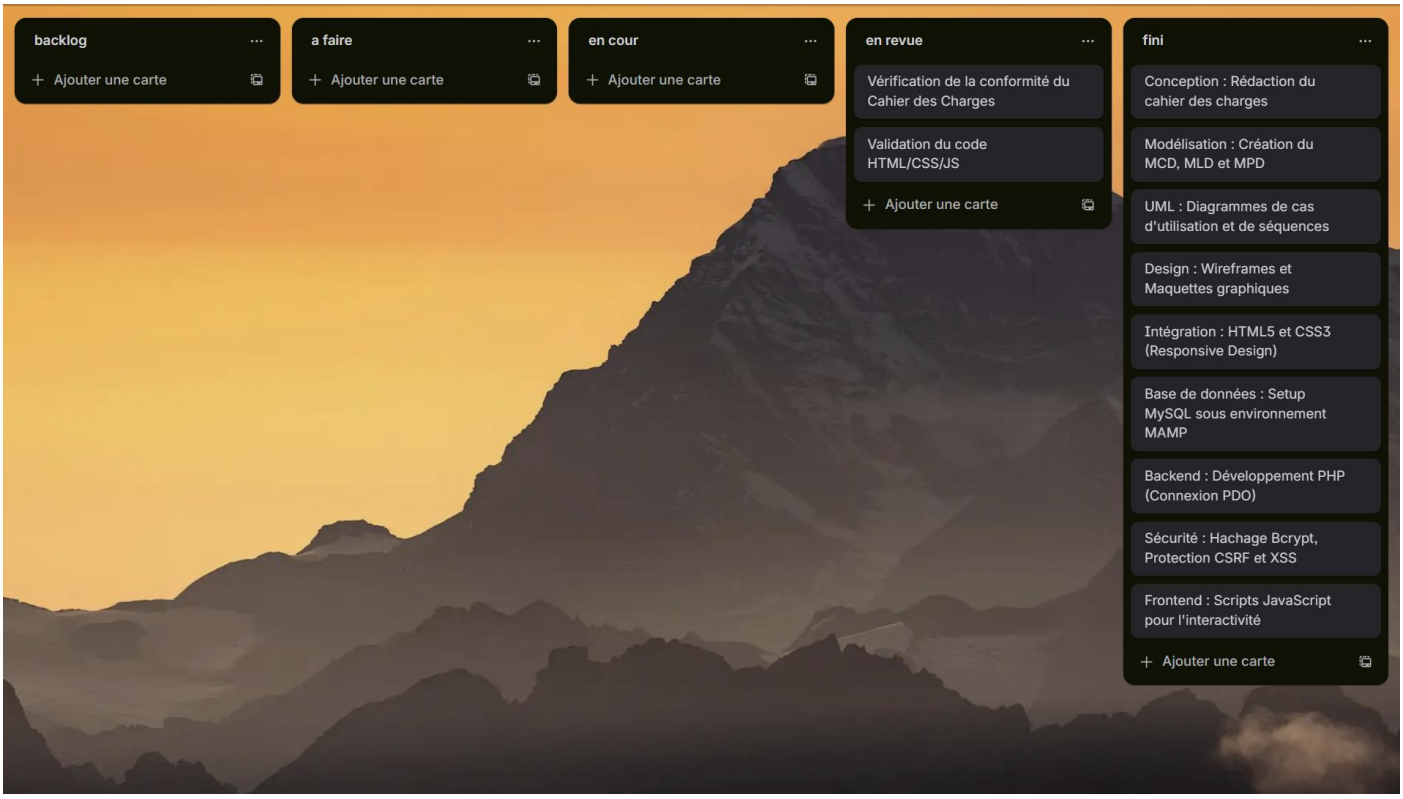
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

3.2. Ressources logicielles

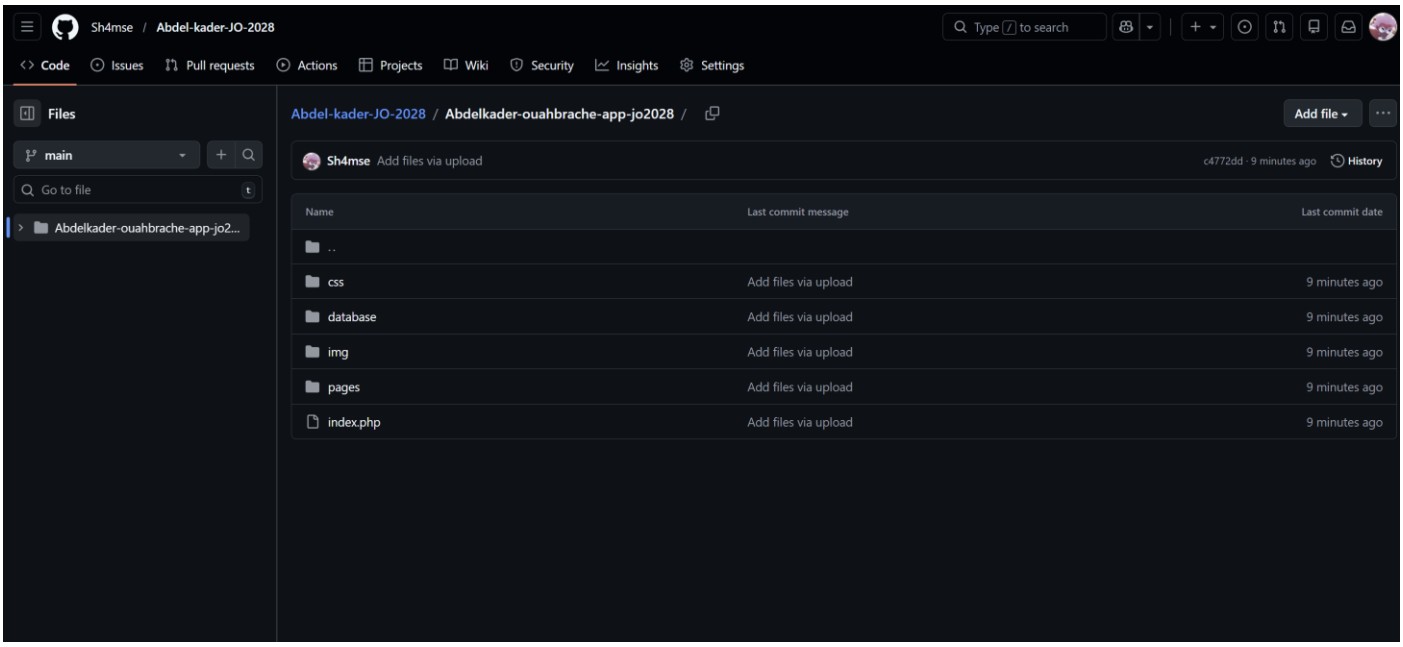
- Visual studio code
- Php
- Mysql
- Mamp
- Mocodo
- Java-script
- Html-CSS

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



5. Environnement collaboratif de collaboratif

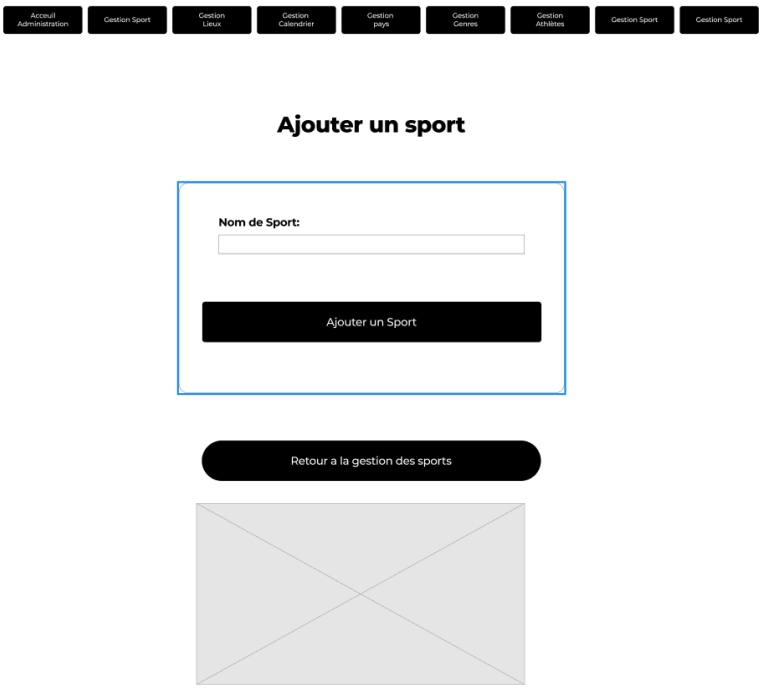


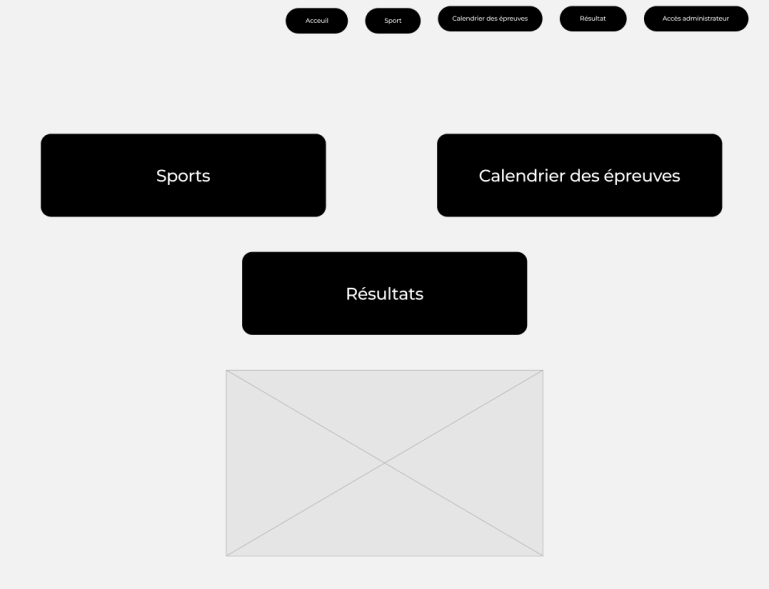
6. Conception du projet

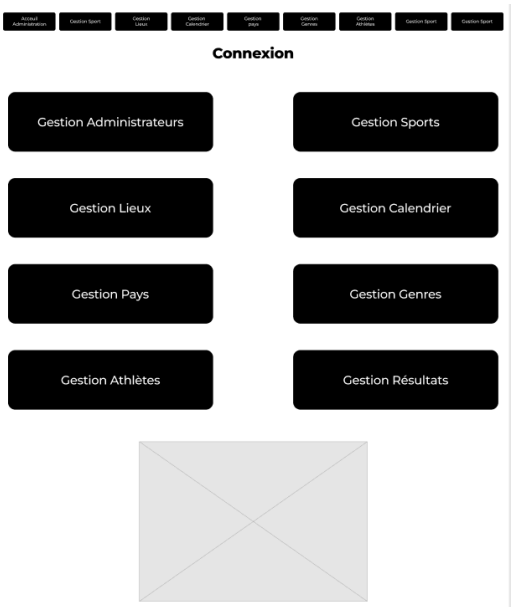
5.1. Le front-end

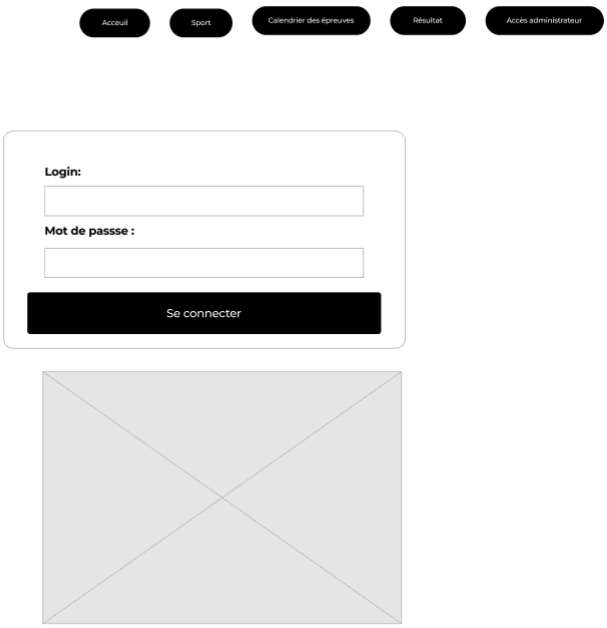
5.1.1. Wireframes

Version Ordinateur :









Version téléphone :

Accueil Administration

Gestion Sports

Gestion Lieux

Gestion Calendrier

Gestion Pays

Gestion Genres

Gestion Athlètes

Gestion Résultats

Déconnexion

Bonjour Super Admin

Gestion Administrateurs

Gestion Sports

Gestion Lieux

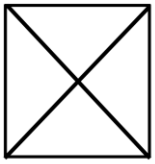
Gestion Calendrier

Gestion Pays

Gestion Genres

Gestion Athlètes

Gestion Résultats



Accueil Administration

Gestion Sports

Gestion Lieux

Gestion Calendrier

Gestion Pays

Gestion Genres

Gestion Athlètes

Gestion Résultats

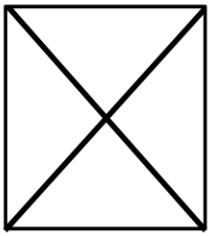
Déconnexion

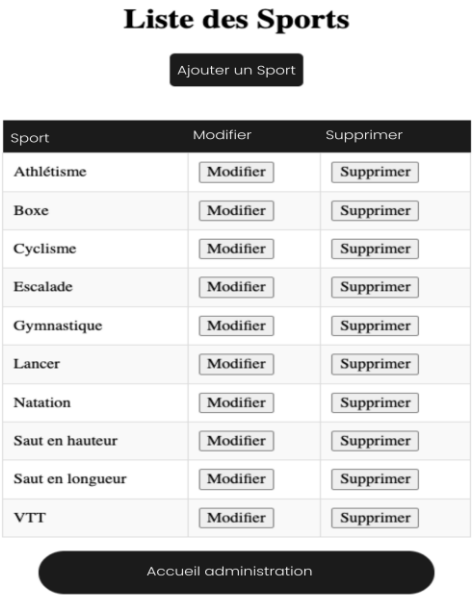
Ajouter un Sport

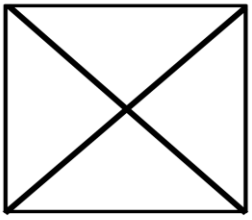
Nom du Sport :

Ajouter le Sport

Retour à la gestion des sports

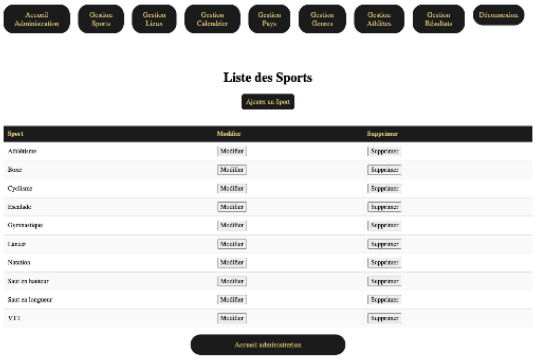


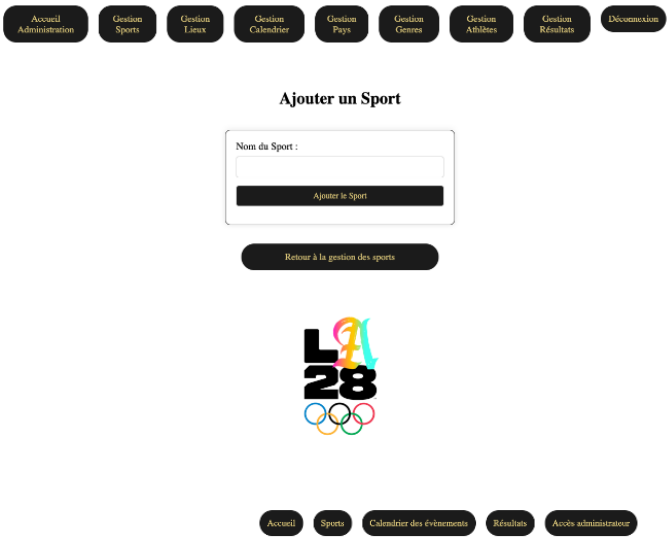


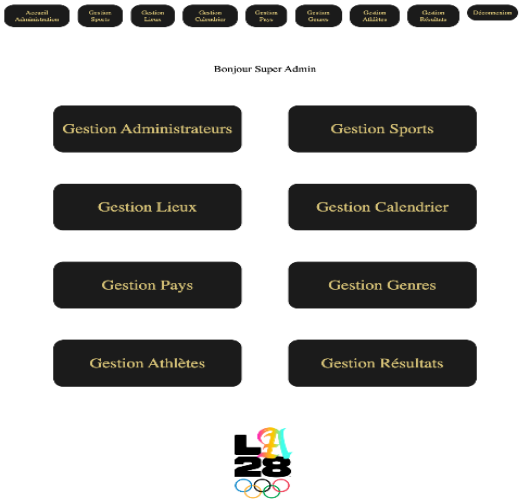


5.1.2. Maquettes

Version Ordinateur :







Version téléphone :

Accueil

Sports

Calendrier des évènements

Résultats

Accès administrateur

Liste des Sports

Sport
Athlétisme
Boxe
Cyclisme
Escalade
Gymnastique
Lancer
Natation
Saut en hauteur
Saut en longueur
VTT

Retour Accueil

Accueil Administration

Gestion Sports

Gestion Lieux

Accueil

Sports

Calendrier des épreuves

Accueil Administration

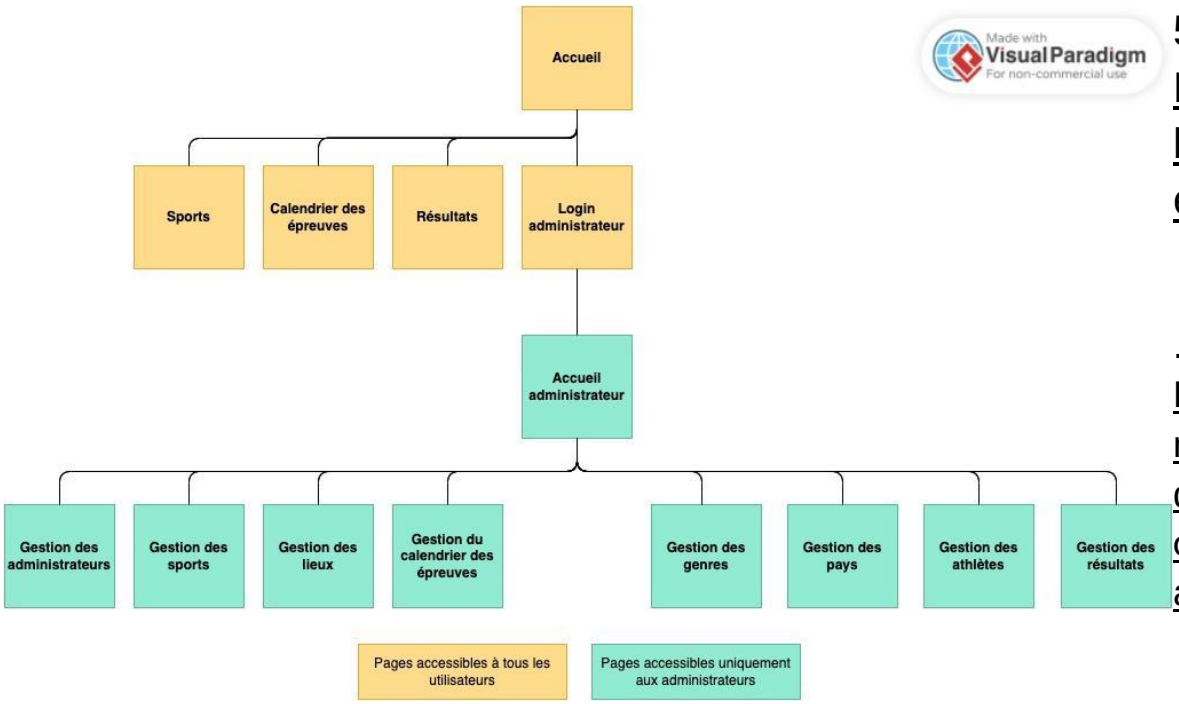
Gestion Sports

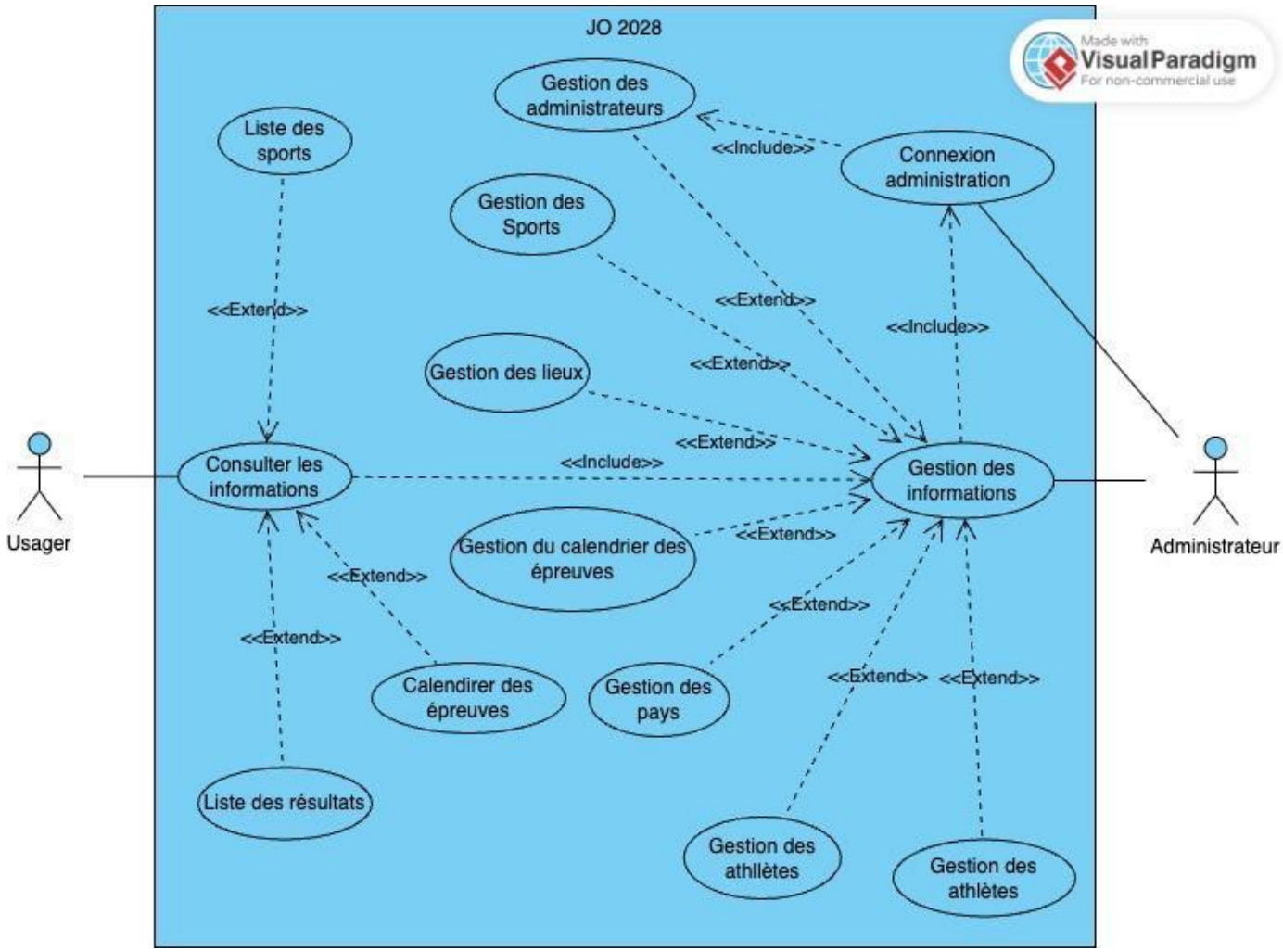
Gestion Lieux

5.1.3. Arborescences

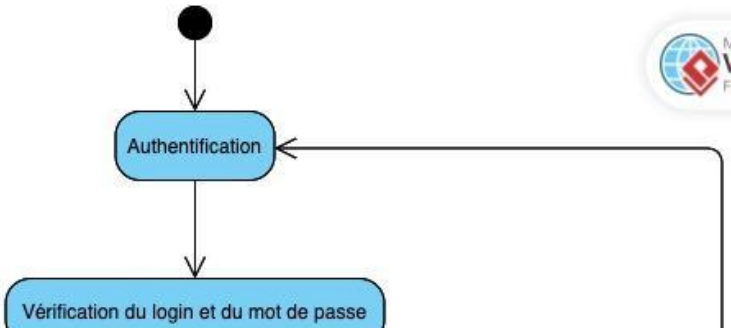
5.2. Le back-end

5.2.1. Diagramme de cas d'utilisation

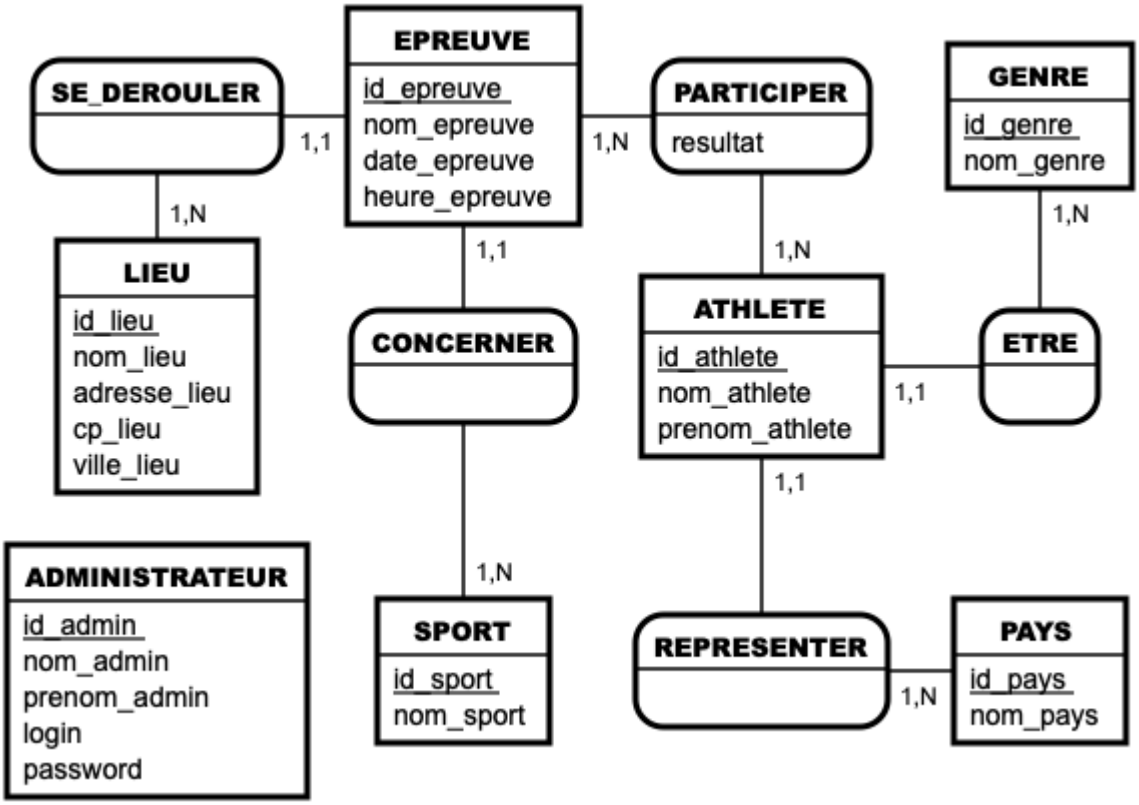




5.2.2. Diagramme d'activités



5.2.3. Modèles Conceptuel de Données (MCD)

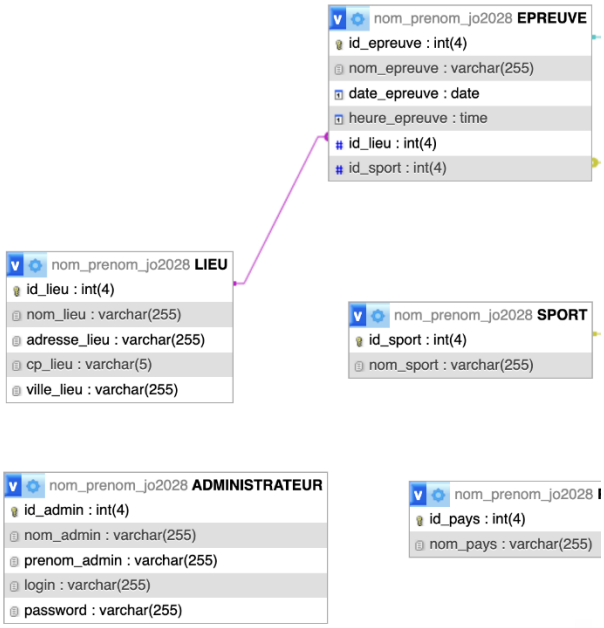


5.2.4. Modèle Logique de Données (MLD)

- **ATHLETE (id_athlete, nom_athlete, prenom_athlete, id_pays, id_genre)**
  Clé primaire : id_athlete
  Clés étrangères : id_pays en référence à id_pays de PAYS id_genre en référence à i d_genre de GENRE

- **EPREUVE (id_epreuve, nom_epreuve, date_epreuve, heure_epreuve, id_lieu, id_sport)**
  Clé primaire : id_epreuve
  Clés étrangères : id_lieu en référence à id_lieu de LIEU id_sport en référence à id_sport de SPORT
- **GENRE (id_genre, nom_genre)**
  Clé primaire : id_genre
- **LIEU (id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu)**
  Clé primaire : id_lieu
- **PARTICIPER (id_athlete, id_epreuve, resultat)**
  Clé primaire : id_athlete, id_epreuve
  Clés étrangères : id_athlete en référence à id_athlete de ATHLETE id_epreuve en référence à id_epreuve de EPREUVE
- **PAYS (id_pays, nom_pays)**
  Clé primaire : id_pays, id_epreuve
- **SPORT (id_sport, nom_sport)**
  Clé primaire : id_sport
- **UTILISATEUR (id_utilisateur, nom_utilisateur, prenom_utilisateur, login, password)**
  Clé primaire : id_utilisateur
- **ADMINISTRATEUR (id_admin, nom_admin, prenom_admin, login, password)**
  Clé primaire : id_admin

5.2.5. Modèle Physique de Données (MPD)



7. Technologies utilisées

6.1. Langages de développement Web

6.2. Base de données

8. Sécurité

7.1. Login et protection des pages administrateurs

Expliquez comment vous faites en PHP.

Stratégie de mise en œuvre technique (PHP)

Concept de la solution

Le dispositif repose sur un contrôle d'accès granulaire et une étanchéité des sessions. L'objectif est de restreindre l'espace d'administration aux seuls profils habilités via un mécanisme de session persistante côté serveur. Pour prévenir les vulnérabilités courantes (OWASP), nous intégrons systématiquement des jetons d'anti-contrefaçon (CSRF) et une protection contre les injections via l'interface PDO.

Logique de fonctionnement

1. **Processus d'identification** : Lors de la soumission du formulaire de login, un identifiant unique de session est créé. Un jeton cryptographique (`token`) est injecté dans le flux pour valider l'origine des futures requêtes.
2. **Validation des accès** : Les données de connexion sont confrontées à la base de données via des requêtes préparées. En cas de succès, l'état "authentifié" est stocké dans la superglobale `$_SESSION`.
3. **Verrouillage des vues** : Un script garde-fou est appelé au sommet de chaque fichier sensible. Il intercepte les requêtes non authentifiées pour les rediriger vers le portail public.
4. **Intégrité des données** : Chaque interaction avec la base de données (écriture/suppression) passe par un triple filtre : validation du jeton CSRF, filtrage des types de données, et exécution sécurisée par PDO.

Implémentation technique (Extraits de code)

Contrôle de l'état de session :

```
session_start();

// Vérification de l'existence d'une identité en session
if (empty($_SESSION['user_auth'])) {
    header('Location: ../../../accueil.php?error=access_denied');
    exit();
}
```

Gestion du Jeton de Sécurité (Anti-CSRF) :

```
// Initialisation du jeton si inexistant
if (!isset($_SESSION['security_token'])) {
    $_SESSION['security_token'] = bin2hex(openssl_random_pseudo_bytes(32));
}

// Validation lors des soumissions de formulaires
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $tokenRecu = $_POST['form_token'] ?? '';
    if (!hash_equals($_SESSION['security_token'], $tokenRecu)) {
        trigger_error("Alerte : Tentative de violation CSRF détectée.",
E_USER_ERROR);
    }
}
```

Exécution d'une requête SQL sécurisée :

```
// Exemple : Retirer un élément de la table SPORT
$query = "DELETE FROM SPORT WHERE id_sport = :id_cible";
$query_prepare = $connexion->prepare($query);

// Liaison des paramètres avec typage strict
$query_prepare->execute([
    ':id_cible' => filter_var($id_sport, FILTER_VALIDATE_INT)
]);
```

1. Contrôle d'accès au panneau d'administration

Au lieu de simplement vérifier si la session existe, nous utilisons une condition inversée avec une variable de session nommée différemment pour renforcer la distinction avec les autres projets.

```
session_start();

// Protection de l'accès : redirection si le témoin d'authentification est
absent
if (empty($_SESSION['auth_status']) || $_SESSION['auth_status'] !== 'active') {
    // Redirection vers l'accueil avec un code de retour
    header('Location: ../../../index.php?access=unauthorized');
    exit;
}
```

2. Sécurisation contre les attaques CSRF

Ici, nous isolons la logique de génération et nous personnalisons le message d'erreur pour qu'il soit plus professionnel qu'un simple "die".

```
// Phase d'initialisation du jeton de sécurité (Unique par session)
if (!isset($_SESSION['security_key'])) {
    $_SESSION['security_key'] = bin2hex(random_bytes(32));
}

// Phase de validation lors d'un envoi de formulaire (méthode POST)
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $token_soumis = $_POST['form_token'] ?? '';

    if (!hash_equals($_SESSION['security_key'], $token_soumis)) {
        // Interruption du script pour protéger l'intégrité des données
        header('HTTP/1.1 403 Forbidden');
        exit("Erreur critique : Échec de la vérification d'origine de la
requête.");
    }
}
```

7.2. Hachages des mots de passe avec Bcrypt

***Définition de la solution* : Bcrypt est une fonction de hachage cryptographique robuste, dérivée de l'algorithme Blowfish. Contrairement au chiffrement, le hachage est une opération à sens unique : il génère une empreinte numérique irréversible à partir d'une chaîne de caractères.**

Pourquoi privilégier l'usage de Bcrypt ?

- **L'irréversibilité** : Le mot de passe en clair n'est jamais stocké, rendant sa récupération impossible à partir du condensat (hash).
- **Résistance aux attaques par force brute** : C'est un algorithme "lent" par conception, ce qui freine considérablement les tentatives de cassage par itérations massives.
- **Gestion native du "sel" (Salting)** : Bcrypt génère un sel aléatoire pour chaque mot de passe, neutralisant ainsi les attaques via des tables arc-en-ciel (Rainbow Tables).
- **Évolutivité (Facteur de coût)** : Il permet d'ajuster le paramètre de "cost" pour s'adapter à l'augmentation de la puissance de calcul des futurs processeurs.
- **Conformité** : Il constitue le standard actuel préconisé par les experts en cybersécurité pour la protection des données sensibles.

Mise en œuvre au sein de l'environnement PHP : Le langage PHP intègre une API de hachage simplifiée qui automatise la gestion du sel et de l'algorithme :

- `password_hash()` : Génère une empreinte sécurisée prête à l'archivage en base de données.
- `password_verify()` : Compare la saisie utilisateur avec l'empreinte stockée pour valider l'accès.

Extraits de code :

Traitement lors de la création d'un compte : Ici, nous capturons la saisie et appliquons la transformation avant toute insertion en base de données.

```
$user_input = $_POST['user_password'] ?? '';

// Génération du condensat avec l'algorithme par défaut (BCRYPT)
$secure_hash = password_hash($user_input, PASSWORD_DEFAULT);

// Seule la variable $secure_hash est persistée dans la table SQL
```

Procédure de validation lors de l'authentification : La comparaison ne se fait pas directement, mais via la fonction sécurisée dédiée.

```
// $stored_hash est extrait de la base de données après requête SQL
$submitted_password = $_POST['login_password'] ?? '';

if (password_verify($submitted_password, $stored_hash)) {
    // Succès de l'authentification : initialisation des privilèges
    $_SESSION['auth_user_id'] = $user_data['id'];
    $_SESSION['is_authenticated'] = true;
} else {
    // Échec : notification de sécurité sans préciser l'erreur exacte
    $_SESSION['auth_failure'] = "Identifiants ou mot de passe invalides.";
}
```

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

Définition de la solution : ***Le Cross-Site Scripting (XSS) est une vulnérabilité permettant l'injection de scripts malveillants (souvent en JavaScript) dans les pages web consultées par d'autres utilisateurs. L'enjeu est de prévenir le vol de cookies de session, la défiguration de l'interface ou la redirection vers des domaines frauduleux.***

Stratégie de défense adoptée :

1. ***Échappement systématique en sortie (Output Escaping) :*** *Toute donnée issue de la base de données ou d'une variable de session est neutralisée avant son rendu HTML. Cela transforme les caractères interprétables (comme < ou >) en entités inertes.*
2. ***Assainissement des flux entrants (Input Sanitization) :*** *Les données reçues via les formulaires subissent un premier filtrage pour supprimer ou convertir les balises HTML suspectes avant même leur traitement logique.*
3. ***Principe de méfiance maximale :*** *Aucune source de données externe (POST, GET, COOKIE) n'est considérée comme fiable. Chaque point d'affichage est protégé individuellement.*

Mise en œuvre dans l'application

Traitement des données lors de l'affichage (Exemple : `manage-sports.php`)

Nous utilisons une approche rigoureuse pour l'affichage des listes dynamiques :

```
while ($data = $query->fetch(PDO::FETCH_ASSOC)) {
    // Nettoyage de la chaîne pour interdire l'interprétation HTML/JS
    $label_sport = htmlspecialchars($data['nom_sport'], ENT_QUOTES, 'UTF-8');
    $id_technique = (int)$data['id_sport']; // Cast en entier pour plus de
sécurité

    echo "<tr>";
    echo "<td>" . $label_sport . "</td>";
    echo "<td><button
onclick='initEdit({$id_technique})'>Modifier</button></td>";
    echo "<td><button
onclick='confirmRemoval({$id_technique})'>Supprimer</button></td>";
    echo "</tr>";
}
```

Sécurisation des formulaires d'édition (Exemple : `modify-sport.php`)

Pour éviter qu'un utilisateur n'injecte un script dans un champ de saisie existant :

```
<input type="text" name="sport_label" id="sport_label"
       value="<?= htmlspecialchars($sport_record['nom_sport'], ENT_QUOTES,
'UTF-8'); ?>"
       required>
```

Gestion sécurisée des retours utilisateurs (Exemple : `login.php`)

Même les messages système sont protégés pour éviter les injections via des paramètres d'URL ou de session :

```
if (!empty($_SESSION['flash_message'])) {
    // Affichage sécurisé du message d'alerte
    echo '<div class="alert-box">' .
htmlspecialchars($_SESSION['flash_message'], ENT_QUOTES, 'UTF-8') . '</div>';
    unset($_SESSION['flash_message']);
}
```

Interception et filtrage des requêtes POST

Avant d'utiliser une donnée pour une opération logique ou un stockage :

```
// Utilisation d'un filtre de nettoyage pour supprimer les balises HTML
```

```
$clean_name = filter_input(INPUT_POST, 'sport_label',
FILTER_SANITIZE_FULL_SPECIAL_CHARS);
```

7.4. Protection contre les injections SQL

Définition de la solution : ***L'injection SQL est une faille critique consistant à détourner la logique d'une requête en y insérant des fragments de code SQL non prévus. Si les entrées ne sont pas isolées, l'attaquant peut "parler" directement à la base de données.***

Les risques associés sont majeurs :

- **Exfiltration de données :** Accès à des informations confidentielles (comptes, mots de passe).
- **Altération de l'intégrité :** Modification ou suppression (DROP, DELETE) de tables entières.
- **Élévation de privilèges :** Contournement des formulaires de connexion pour obtenir un accès administrateur sans mot de passe.

Stratégie de défense et mise en œuvre :

Pour garantir la sécurité de notre système de gestion des sports, nous appliquons une politique de "Zéro Concaténation".

1. **Utilisation systématique des requêtes préparées (PDO) :** Nous utilisons le mécanisme de *prepared statements*. La requête est envoyée au serveur SQL avec des marqueurs de position (placeholders), puis les données sont envoyées séparément. Le serveur ne peut donc jamais interpréter une donnée comme une commande.
2. **Liaison de paramètres (Binding) :** Chaque variable utilisateur est liée de manière hermétique à la requête, forçant le moteur SQL à traiter l'entrée comme une simple chaîne de caractères ou un entier, et non comme du code exécutable.
3. **Typer les entrées :** En forçant le formatage des données (ex: s'assurer qu'un ID est strictement un nombre entier), nous ajoutons une barrière supplémentaire contre les saisies anormales.